Here are **Intermediate- level only** questions (40 total: Lists, Tuples, Sets, Dictionaries), each with a **helpful tip**.
All questions are moderately challenging but still accessible for learners at an intermediate tier.

Part A — Python Lists (10 Intermediate-Level Questions)

1. **Create a list comprehension that returns the squares of only the even numbers from 0–20.**
   Tip: Use an if condition inside the comprehension.
2. **Given** nums = [3, 1, 4, 1, 5, 9]**, sort the list without modifying the original.**
   Tip: Use sorted() instead of .sort().
3. **Remove duplicates from a list while preserving the original order.**
   Tip: Track seen values in a new list.
4. **Flatten the nested list** [[1, 2], [3, 4], [5]] **into a single list using a list comprehension.**
   Tip: Use a nested loop inside the comprehension.
5. **Given** names = ['alice', 'Bob', 'charlie', 'DAVID']**, sort them alphabetically but ignore case.**
   Tip: Use key=str.lower.
6. **Replace items from index 2–4 in a list with** [100, 200] **using slice assignment.**
   Tip: Use a[2:5] = [...].
7. **Write a program to find all indices of a value in a list (e.g., all indices of** 7**).**
   Tip: Use enumerate.
8. **Create a new list containing only elements that appear exactly once in the original list.**
   Tip: Use list.count() inside a comprehension.
9. **Rotate a list right by one position (e.g.,** [1,2,3,4] → [4,1,2,3]**).**
   Tip: Use slicing: lst[-1:] + lst[:-1].
10. **Split a list into two lists: one with even numbers, one with odd numbers.**
    Tip: Create two comprehensions.

Part B — Python Tuples (10 Intermediate-Level Questions)

1. **Convert the list** [1, 2, 3, 4] **into a tuple and then unpack it into four variables.**
   Tip: Use tuple() and simple unpacking.
2. **Given** t = (('a', 1), ('b', 2), ('c', 3))**, create a list of all second elements.**
   Tip: Use a comprehension: x[1].

3. **Write a function that returns multiple values (sum, min, max) using a tuple.**
   Tip: Return (..., ..., ...) and unpack later.
4. **Combine two tuples** (1, 2, 3) **and** (4, 5) **then convert the result to a list.**
   Tip: Use + to join them.
5. **Given a tuple of numbers, find the element with the highest frequency.**
   Tip: Loop through unique items using set(t).
6. **Check if two tuples contain the same elements regardless of order.**
   Tip: Compare sorted(tuple) values.
7. **Extract the last three items from a tuple using slicing.**
   Tip: Use negative indexing: t[-3:].
8. **Concatenate a tuple with itself three times (repeat operation).**
   Tip: Use tuple * 3.
9. **Convert a nested tuple** ((1,2),(3,4)) **into a flat tuple** (1,2,3,4)**.**
   Tip: Use a comprehension inside tuple().
10. **Store coordinates in tuples and calculate the Manhattan distance.**
    Tip: Use absolute difference formula: abs(x1-x2) + abs(y1-y2).

Part C — Python Sets (10 Intermediate-Level Questions)

1. **Given two sets, find elements that are in the first set but not the second.**
   Tip: Use the - operator.
2. **Find common items between three sets using intersection.**
   Tip: Use set1 & set2 & set3.
3. **Given a sentence, return all unique words in lowercase.**
   Tip: Split the string → lowercase → convert to set.
4. **Convert a list with duplicates into a set, then back to a sorted list.**
   Tip: Use sorted(set(list)).
5. **Check if one set is a strict subset of another.**
   Tip: Use < operator.
6. **Use a set comprehension to collect all squares of numbers from 1–15 that are divisible by 3.**
   Tip: Write {x*x for x in ... if x % 3 == 0}.
7. **Count how many duplicate values exist in a list using sets.**
   Tip: Compare lengths: len(list) - len(set(list)).
8. **Write a program to remove all vowels from a string using a set.**
   Tip: Use a vowel set and filter characters.
9. **Find the symmetric difference between two sets.**
   Tip: Use the ^ operator.

10. **Check if two strings are anagrams using set comparison (unique characters only).**
    Tip: Compare set(str1) with set(str2).

Part D — Python Dictionaries (10 Intermediate-Level Questions)

1. **Count word frequencies in a sentence and store the results in a dictionary.**
   Tip: Use d[word] = d.get(word, 0) + 1.
2. **Invert a dictionary where all values are unique.**
   Tip: Swap key and value in a loop.
3. **Merge two dictionaries where second dictionary overrides first.**
   Tip: Use {**dict1, **dict2} or dict1 | dict2 (Python 3.9+).
4. **Group words by their first letter into a dictionary of lists.**
   Tip: Use setdefault.
5. **Filter a dictionary to keep only entries with values greater than 50.**
   Tip: Use a dictionary comprehension.
6. **Given a nested dictionary, safely access a deeply nested key.**
   Tip: Chain .get() calls with default {}.
7. **Write a dictionary comprehension that maps numbers 1–10 to their cubes.**
   Tip: {x: x**3 for x in range(1,11)}.
8. **Find the key with the highest value in a dictionary.**
   Tip: Use max(d, key=d.get).
9. **Combine two lists into a dictionary (keys from first list, values from second).**
   Tip: Use zip().
10. **Remove all keys from a dictionary whose values are** None**.**
    Tip: Check value before adding to new dict.